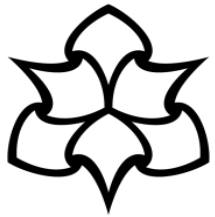
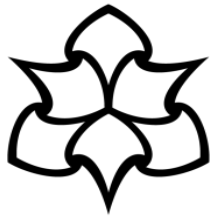
Manchester Metropolitan
University



Full-Stack Web Development

Lecture 2: Introduction to NodeJS/ExpressJS

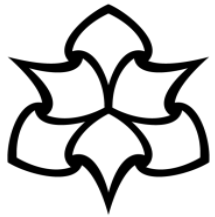
Ashley Williams
Ashley.Williams@mmu.ac.uk



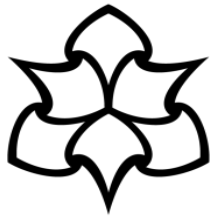
Learning Objectives

- To understand why JS is the way it is!
- To understand the basics of NodeJS, NPM, ExpressJS
- To learn how to start writing server applications

Manchester Metropolitan
University

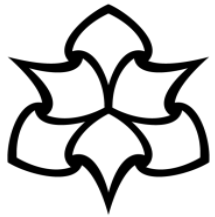


The Assignment



The Assignment

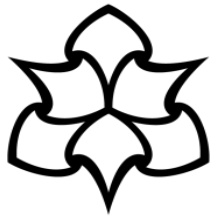
- Assignment Spec
- API Spec
- Starter code
- How the assignment is marked
- The tests



Homework!

- Get the assignment running on your laptops before the next lab
- Familiarise yourself with the API spec
  - How?
    - Run through scenarios in your head
    - Sketch out some wireframes on a piece of paper so that you can visualise the frontend app. Where do each of the end-points get hit?

Manchester Metropolitan
University



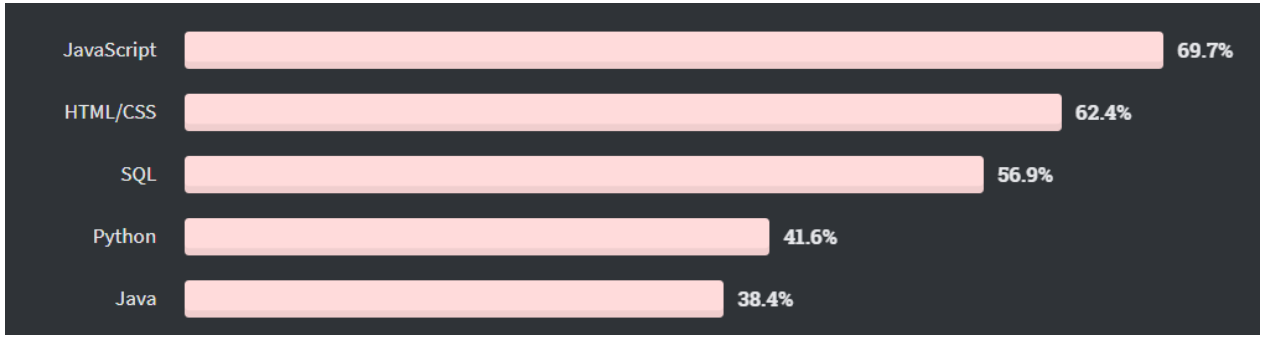
JavaScript

Manchester Metropolitan
University

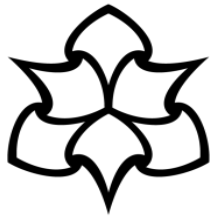
The most commonly used programming language in the world!

- Stack Overflow 2020 Developer Survey

- Web Development
  - Used by 95.2% of all websites
- Game Development
- Mobile Apps
- Building Web Servers

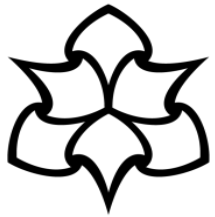


- https://insights.stackoverflow.com/survey/2020#technology-programming-scripting-and-markup-languages-professional-developers



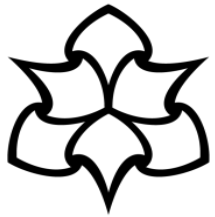
Timeline

- Early 90's – The Browser Wars (NetScape vs MicroSoft)
  - Companies fighting for market share of the web
  - NetScapes mistake?
    - https://www.joelonsoftware.com/2000/04/06/things-you-should-never-do-part-i/
- 1995 – Brandon Eich (working for NetScape) developed a new scripting language in 10 days
  - Called Mocha, but later became JavaScript
- 1997 – JS is growing and needs to be properly managed and maintained
  - NetScape hands it over to European Computer Manufacturers Association (ECMA)
  - ECMA Script is a standard for programming languages (JS, JScript, ActionScript)
- 2005 – Ajax is released, allowing dynamic server calls without refreshing (the start of Web2.0)



Timeline

- 2009 – CommonJS project sets out to promote JS development outside of the browser using modules to package code and functionality (paved the way for NodeJS)
- 2012 - TypeScript is publicly released
  - A superset of JS which adds optional static typing – transpiles to JS
- 2015 – ECMAScript 6 is released with a bunch of major upgrades
  - It is also referred to as ES6 or ECMAScript2015 – confusing??
  - ES6 == ES2015, ES7 == ES2016, but ES5 was 2009! And ES4 was 1999!
- They standardise on yearly releases and just call it ES2022, ES2023, ES2024
- **But why am I telling you this?**



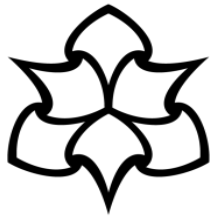
Why?

- Well we will be using NodeJS to write our server – one language to do everything!
  - Previously unheard of before the CommonJS project

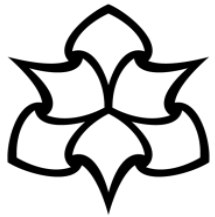
- You will hear people talking about ES6, ES7, TypeScript etc
  - Now you know what they're on about!

- **Which version of JS do I use?**
  - Well it is more about looking at what features you can use on what browser, and what you want to support - https://caniuse.com/es6

Manchester Metropolitan
University



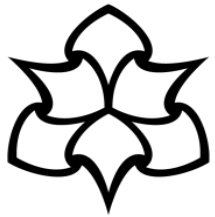
NodeJS



What is NodeJS?

- “As an asynchronous event-driven JavaScript runtime, Node.js is designed to build scalable network applications.” - https://nodejs.org

- Open Source
- Server Environment
- Uses JS on the back-end

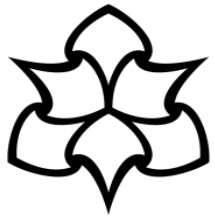


What is NodeJS?

- NodeJS is asynchronous, but JS is single-threaded…and Node JS is JS
  - Huh?

PHP (blocking code)	Node (non-blocking code)
1. Sends a task to the file system	1. Sends a task to the file system
2. Waits while the task completes	2. Is ready to handle the next request
3. Returns the content to the client	3. When the task has completed, returns to the client
4. Is ready to handle the next request	

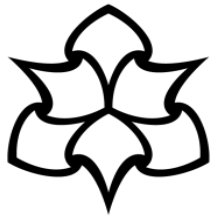
Taken from: https://www.w3schools.com/nodejs/nodejs_intro.asp



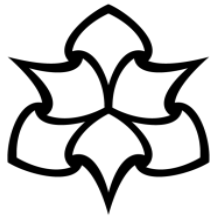
How does it work?

- Built on Chromes V8 engine - JS runtime, web APIs, event loop, call-back queue
- Rather than block the stack, tasks are handed off to the Web APIs (essentially can be thought of as other threads)
- On completion, the call-back function is sent to the call-back queue to be executed
- The event loop checks for when the stack is clear, and then sends the next function in the call-back queue to be executed
- This video explains it really well with animations: “What the heck is the event loop anyway?” - https://www.youtube.com/watch?v=8aGhZQkoFbQ
- **Still don’t understand?** – don’t worry, its more detail than you need for this Unit

Manchester Metropolitan
University

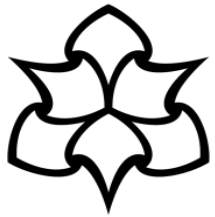


Node Package Manager (NPM)

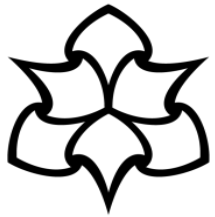


What is NPM?

- Installed at the same time you install NodeJS
- Remember this:
  - *2009 – CommonJS project sets out to promote JS development outside of the browser using modules to package code and functionality (paved the way for NodeJS)*
- NPM is used for installing/uninstalling external packages of code/libraries to use within your project – why reinvent the wheel?
- Packages are installing into the “node_modules” directory ready to be imported into your project
- Install new packages with: “npm install --save <library name>”
  - The “--save” adds the dependency to your **package.json** file

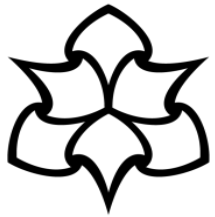


Package.json



But, what is a package.json file?

- All NPM packages (including all Node apps) have a file in their root directory called package.json
- It holds all of the metadata for the project
  - Name
  - Maintainer
  - Version
  - Test information
  - List of dependencies – NPM packages needed to run



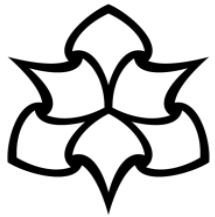
A simple package.json file

- Remember the moviedb API from last year?

- Express and Joi will both be used in your assignments

17 lines (17 sloc) | 359 Bytes

```
{
  "name": "moviedb",
  "version": "1.0.0",
  "description": "MovieDB API",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Ash Williams",
  "license": "MIT",
  "dependencies": {
    "express": "^4.17.1",
    "joi": "^17.2.1"
  }
}
```



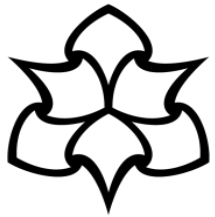
The assignment package.json

- Slightly more going on…
  - Test scripts
  - More dependencies
    - This will grow as you find more libraries to use

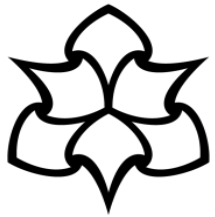
22 lines (22 sloc) | 562 Bytes

```
{
  "name": "blog-engine-server",
  "version": "1.0.0",
  "description": "For the full-stack web development assignment",
  "main": "server.js",
  "scripts": {
    "test": "mocha ./tests/test.*.js --reporter spec --log-level=warn --timeout 10000",
    "dev": "nodemon server.js"
  },
  "author": "Ash Williams",
  "license": "MIT",
  "dependencies": {
    "body-parser": "^1.20.0",
    "chai": "^4.3.6",
    "chai-http": "^4.3.0",
    "express": "^4.18.1",
    "mocha": "^10.0.0",
    "morgan": "^1.10.0",
    "nodemon": "^2.0.19",
    "sqlite3": "^5.0.9"
  }
}
```

Manchester Metropolitan
University



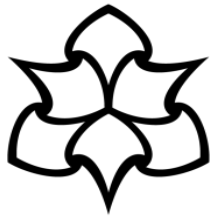
ExpressJS



What is ExpressJS?

- Fast, unopinionated, minimalist web framework for Node.js - https://expressjs.com/
- An NPM package – installed with “npm install --save express”
- Provides you with the ability to create web servers and APIs easily and flexibly
- *“Perhaps the most important NPM package that exists”* – Ash, 2024

Manchester Metropolitan
University

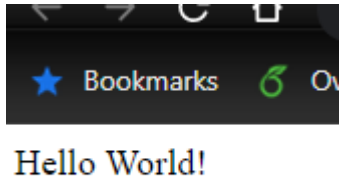


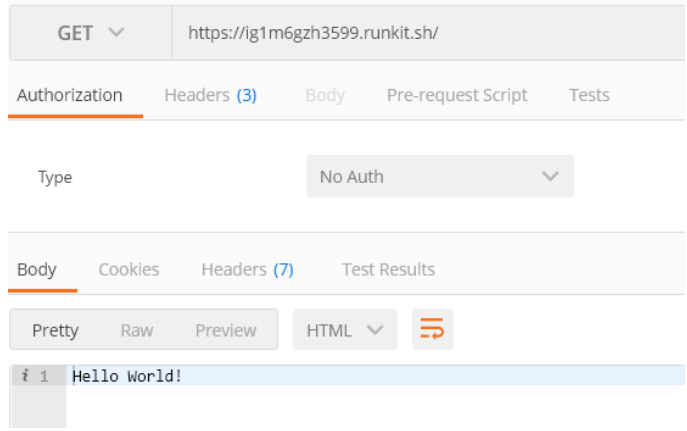
Creating our first server application

Hello World

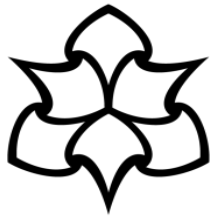
- “require” pulls in the package from our node_modules directory
- “express()” initiates the express app
- Lines 5,6,7 says that when a get request is send to the servers root (“/”), respond with “Hello World!” – this is called a **route** or an **end-point**
- Lines 9,10,11 sets the server listening on a specific port and gives a message when its ready
- **What is a port?**
  - IP address refers to a device
  - Ports can be used by a specific application on the device

```
1  const express = require('express' 4.18.1 )
2  const app = express()
3  const port = 3000
4
5  app.get('/', (req, res) => {
6    res.send('Hello World!')
7  })
8
9  app.listen(port, () => {
10   console.log(`Example app listening on port ${port}`)
11 })
```

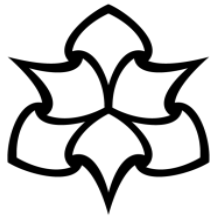




Manchester Metropolitan
University



Assignment starter code

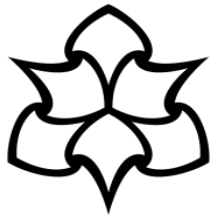


Assignment starter code

- Pretty much the same…
- Some new libraries
  - Morgan: pretty logging to the console
  - BodyParser: helps parse JSON
- A simple start route
- And a default route at the bottom
- You add the other routes in the middle

34 lines (24 sloc) | 648 Bytes

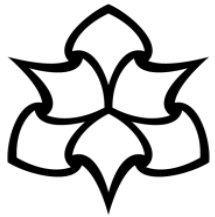
```
1   const express = require("express")
2   const morgan = require('morgan')
3   const bodyParser = require("body-parser");
4
5
6   const app = express()
7
8   // Server port
9   const HTTP_PORT = 3333
10
11  // Start server
12  app.listen(HTTP_PORT, () => {
13      console.log("Server running on port: " + HTTP_PORT)
14  });
15
16  // Logging
17  app.use(morgan('tiny'));
18
19  // Body parser
20  app.use(bodyParser.urlencoded({ extended: false }));
21  app.use(bodyParser.json());
22
23  // Root endpoint
24  app.get("/", (req, res, next) => {
25      res.json({"status":"Alive"})
26  });
27
28  // Other API endpoints: Links go here...
29
30
31  // Default response for any other request
32  app.use(function(req, res){
33      res.sendStatus(404);
34  });
```



Today's lab

- Get to grips with creating basic ExpressJS apps in Node
- Write, and interact with your first web server and API
- Create an API that interacts with an array
  - (like the MovieDB API, but different)

- Assignment Checklist:
  - Familiarise yourself fully with the assignment API
  - Download and run the starter code project



Questions?